

高德前端框架思考与展望

引言

悟以往之不谏,知来者之可追

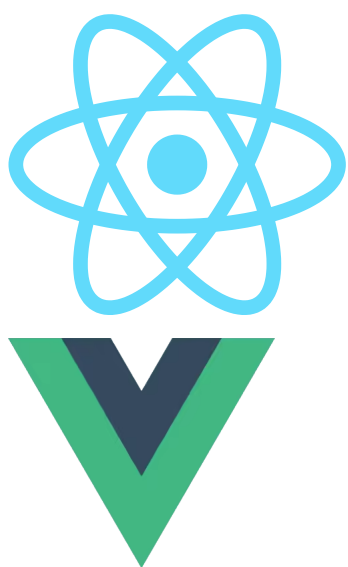
作为《AMap前端技术系列》之一,编写本文时也难得停下脚步回顾、整理AMap业务高速发展这五年来前端框架的发展历程,梳理在不同业务阶段对框架选择的思考过程,以及对未来的展望。

起航:寻找AMap动态化前端框架

合格的开发者知道怎么做,优秀的开发者知道为什么这么做

在2017.4这个时间节点上,前端框架生态欣欣向荣:Backbone、Ember等虽老而弥坚但React、Vue、Angular已成三足鼎立之势、并且触手从web端延展到了native&hybrid(RN、Weex、Ionic...);AMap内部也摸索出了适合地图业务的《动态化跨端渲染引擎-AJX》。

底层渲染引擎已经能力完备,前端面临的第一个问题就是应该选择什么前端框架?来打造AMap的"x-native"。





摸索:第一代框架AJXFramework

合适的选择不一定是最好的方案

当时面临的情况是：

- AJX渲染引擎有一套和浏览器类似的编程接口，但是它的dom操作又有明显限制和区别
- 业务开发和基建几乎同时进行，并且要在短时间内灰度上线
- 试水AJX业务依然以前端团队为主
- 先期试水的业务复杂度有限
- AJX引擎对业务层来说还比较黑盒，前端框架的适配一定是频繁的

基于以上因素，在试水业务的开发框架的选择上，确立了几个原则：

- **不阻塞业务开发**：如果接入主流框架，需要深度改造vue或编写react-ajx来替换react-dom或者深入改造一些react-like，但在当时的项目压力下是来不及的，但同时必须有一套中间框架来做收口和底层引擎屏蔽
- **最小可用**：框架有基本的组件化能力和结构约束即可，不能有太多性能负担
- **框架白盒化**：把变动因子尽可能缩小，如果框架和引擎都是黑盒，问题的排查链条将会非常长

最后的方案：基于已有的JSX编译能力+一个简单的runtime框架补充：**AJXFramework**

- 基本的页面骨骼、生命周期定义
- 业务可以用JSX语法实现基本组件化
- 可以让业务可以从"组件"这个级别进行dom的增、删、改。

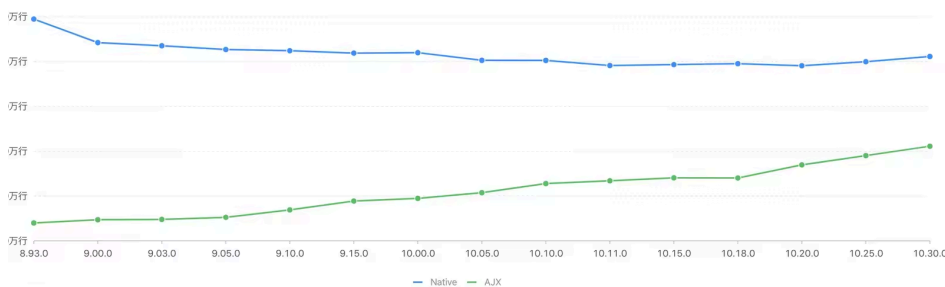
实际上这套基础框架更像WebComponent,基于这套基本能力迅速的落地了POI详情页和Feed流业务，并且顺利上线，算是对AJX渲染能力进行了初步验证。

风暴:剧增的AJX业务量，前端框架小马拉大车

蒸汽机终要代替马车

在经过几个灰度版本验证后，AJX在动态化、性能上相比传统的native和h5有综合性的优势。结合 [高德业务](#) 快速发展的大背景，AJX作为后续业务开发的首选方案趋势越来越明显，

开发业务的团队也不是纯前端同学的小范围，AJXFramework每天产生的问题和功能的不完备已经让前端同学支撑起来感觉力不从心，也让客户端同学开发起来十分难受。



AJX和Native代码量趋势对比（具体数字脱敏）

和AJX业务遍地开花相反，**AJXFramework**作为一个折中方案，在"大前端"的团队矩阵下，已经逐渐成为一种落后的生产力工具：

- **AJXFramework只是一个UI层解决方案**：在AMap大多数重逻辑的业务中，逻辑和UI混杂的问题非常严重
- **AJXFramework无生态可用**：在框架生态的延展上，几乎都不可避免的要造轮子，而经验告诉我们，学习轮子，而不要造轮子！不要造轮子！不要造轮子！
- **AJXFramework不是声明式的**：虽然在语法采用JSX但AJXFramework本身没有脱离dom操作的模式，无论接入mobx还是redux，当项目足够大时难以避免项目中充满大量零散的"p"对象（mvp），并且数据的流向也越来越难以琢磨，一个数据的改变导致UI的状态是什么根本无法预知，这点和大项目使用backbone的问题是一致的。

毫无疑问，接入主流前端生态已经迫在眉睫。

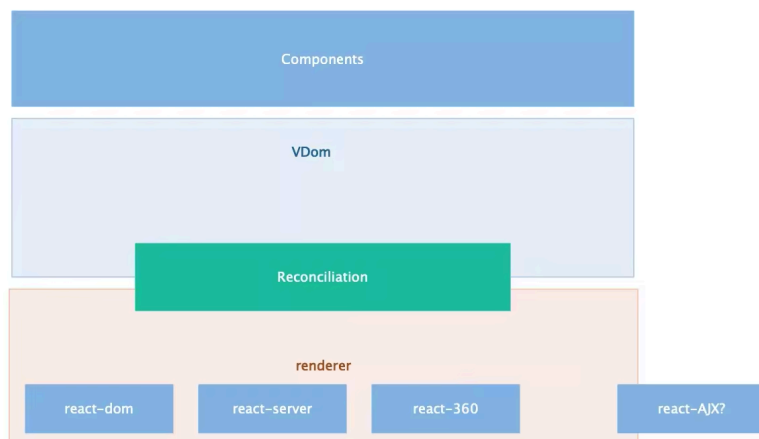
黎明:第二代框架FOX

没有什么问题是不能通过增加一层抽象解决的，如果有那就抽象两层

vue or react （react-like）？

因为react本身只关心抽象组件的组织与调度，而不关心渲染容器具体是谁，在render时引入不同renderer来实现不同宿主环境的渲染即可，我们参考react-dom实现react-ajx即可（react-conf 2019也有一个专门关于自定义renderer的speech，相信react将会出现在越来越多的小程序runtime解决方案和各种奇怪的终端设备上）。

vue也类似这里不再展开。



但我们没有选择react或是vue，而是选择了rax（0.4版本）

- 优势：

- 首先rax并不是又一个react-like，它只是将react作为一种上层dsl对齐。并且从设计之初就定位于跨渲染容器的解决方案，所以它的vdom、调度和host分离的非常好，并且给了很明确的抽象，我们用AJX宿主API去实现driver的抽象即可，基本可以做到0侵入rax本身
- 相比react，它非常适合无线端
- 大部分react生态可以无缝接入（rax-redux、dva、mobx等）
- 集团内部团队已经有大规模使用，生态一定会紧跟react

- 问题：

- 同步的setState(0.x版本，1.0.xx版本已改为异步)，可能会带来一些频繁render问题，但可以通过约束写法规避。
- 本身无法通过react 100%的case（比如ref的卸载和更新顺序），但对于我们来说并不重要，因为AMapC端的业务性质对于复用一些react ui库或其他react生态没有强诉求。

rax适配

基于rax以上优势，看起来只要用ajx接口实现rax-driver api定义即可



复制代码

```
const Driver = {
  createBody() {},
  ...
  createElement(type, props) {},
}
```

```

appendChild(node, parent) {},
...
addEventListener(node, eventName, eventHandler) {},
setAttribute(node, propKey, propValue) {},
...
};

```

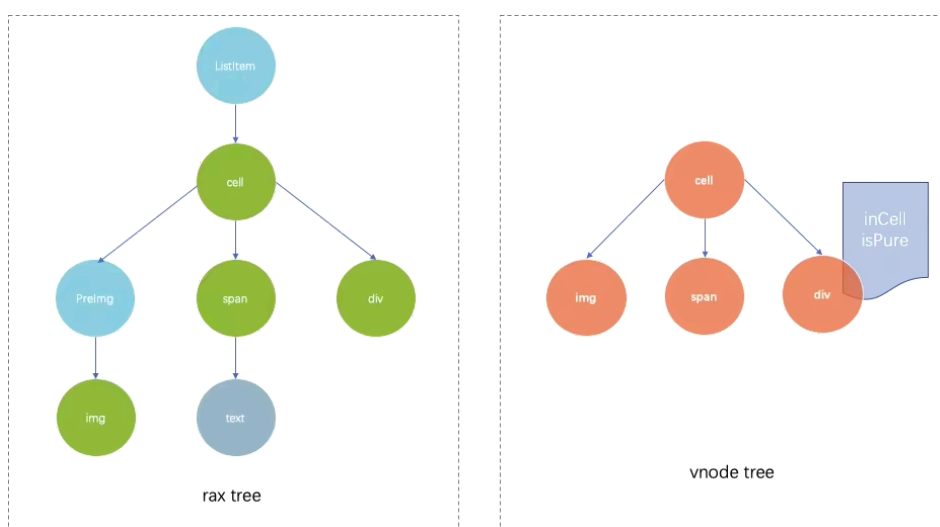
在适配当中也遇到以下困难：

- AJAX渲染引擎cell的子、孙节点是不能被修改的
- AJAX渲染引擎的createElement+append的性能要远远低于innerHTML，所以必须要用tree的方式整体拼装好
- AJAX无法用一个节点作为Empty节点，不同的tag下需要用不同的Empty节点
- ...

解决这些问题，第一直觉是去fork rax修改reconciliation及diff patch过程。但这样会大量侵入rax源码，并且侵扰了reconciliation的正常生命周期，后续接入任何生态将非常困难。

那就再增加一层抽象：**vnode**，让调度后的dom patch都作用于**vnode**，vnode再去化解这些差异

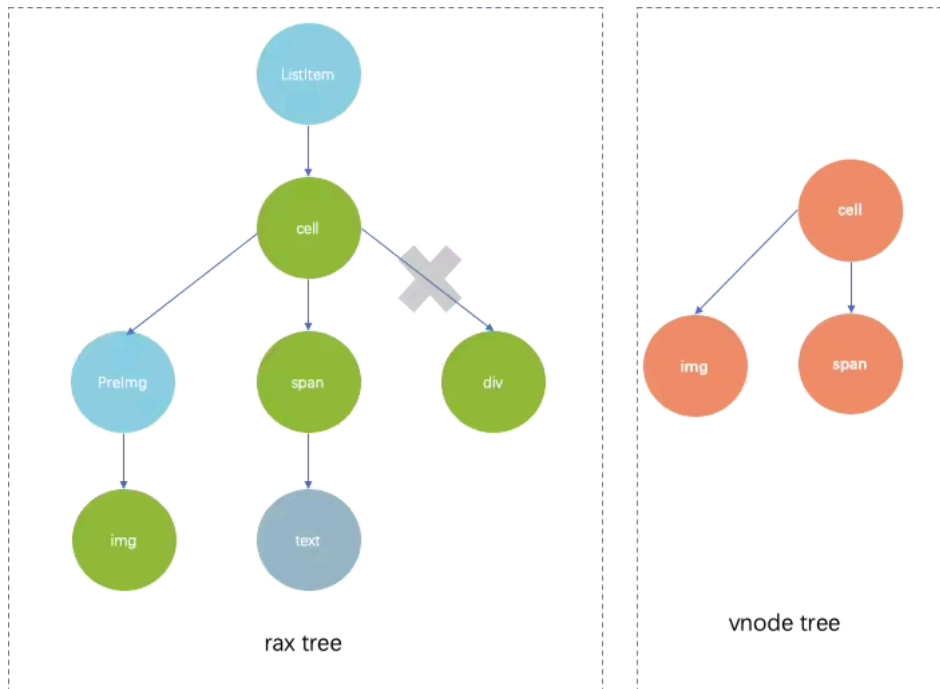
- 挂载时：

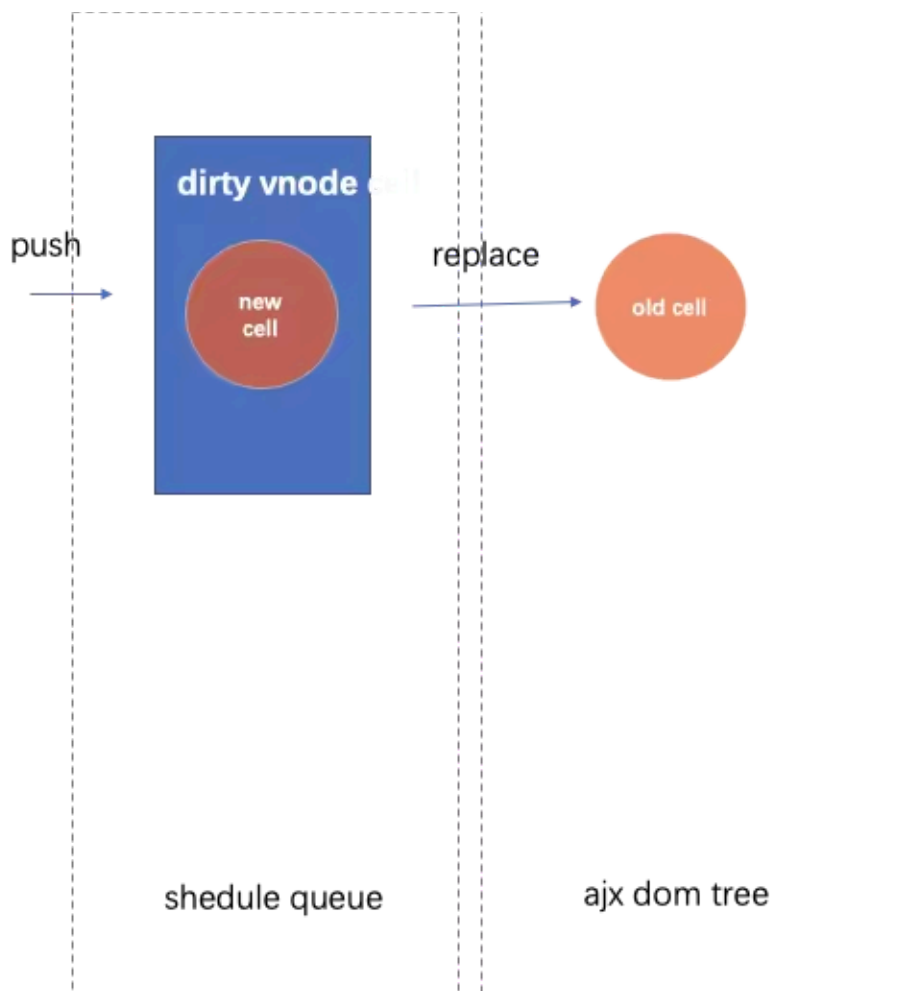


- 以上步骤：

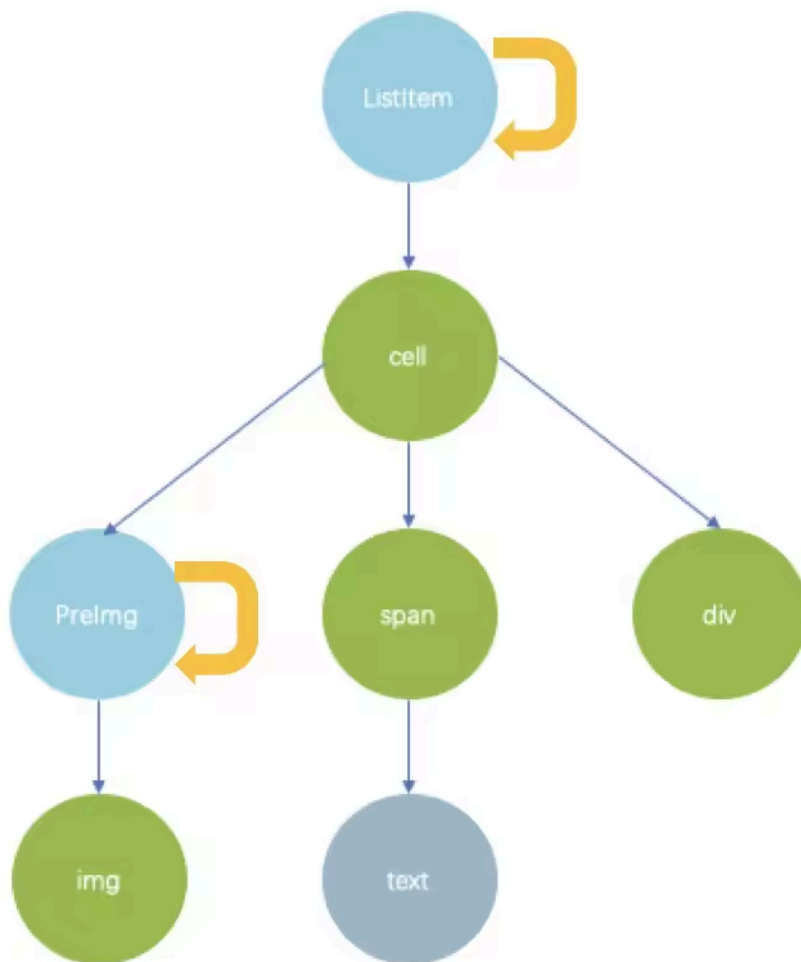
- createElement创建的是vnode节点，而非真正的dom节点，并且在逐层append中形成vnode tree
- 通过判断自己和祖先的tag type来表示在cell内还是外
- 最后，从vnode节点遍历拼成完整xml，一次html后挂载在对应的ajax parentNode上

- 更新时：





- 以上步骤：
 - 如果更新发生在cell外直接放行，如果在cell内修改对应vnode，并且标脏cell，推到调度队列
 - vnode再通过自己的调度机制在rax的committed阶段，识别出dirty cells，重新拼装好完整的cell整体替换
- 生命周期补偿：



rax tree

- 以上步骤：
 - 因为cell节点和子孙节点都是新节点，所以要重新做一遍做unmount&mount生命周期的补偿
 - 首先找到包含cell的最closest的Component再一路向下找到所有的Component实例，做unmount&mount的生命周期的补偿（因为cell和子孙节点都是新的了）
 - 注：如果ListItem还有其他子节点，可能导致影响其他其他节点重走生命周期，所以cell一般都单独包一个Cell Wrapper节点

空节点”、“事件处理”等差异的处理，这里就不再展开

数据流

数据流方案采用了集团的dva：

- 极简语法，将所需一切打包就位，开箱即用

- 副作用、状态突变在语义和语法上都进行了区分，不需要有函数式编程的背景就能写出非常规范的代码

我们也对dva做了定制化，解除了dva和redux-saga的绑定

redux-saga在AMap下的IO和执行时间在Iphone 5s上几乎要花掉**80ms**，在保持dva核心语法的前提下替换成了自己开发的一个轻量的异步中间件（当然是以牺牲effect可以测试的代价下）。

最终针对AJX引擎的页面生命周期进行抽象、框架绑定，将以上进行打包形成了AMap的前端开发体系-**FOX**

浪潮

将至已至

在文章的最后，我来大胆预测一下前端框架未来的趋势，以及我自己将储备什么来迎接一下波浪潮。

**** 越来越多的框架会从取悦开发者到服务于用户体验 ****

用户体验一开始就会作为构建框架的基因之一（如fiber，当然fiber也不是银弹），而不是基于框架而衍生出的各种性能优化方案（如：ssr、nsr），框架内部只会变得越来越复杂相应的学习成本也会越来越高，这就要求框架深度使用者或支撑业务的架构同学对各种编程范式及计算机的运行本质有一定了解。

**** 低代码工具和元框架越来越多 ****

因为学习曲线的上升，基于框架之上的抽象dsl和低代码工具会越来越多，在复杂的框架本身重新定义一套有约束的简单抽象拉低业务开发者的使用成本。

**** 前端框架的技术品味从“混合范式”将会越来越偏向于“响应函数式” ****

因为“响应函数式”是最逼近人类解构这个世界的抽象方式：

比如当我们骑车忽然车链子掉了，我们去上车链子的同时，还要躲避马路上来往的车辆。我们脑子中不会映射出和当前现实世界一一映射的对象，而是根据我们已有经验（函数），随着外界不同刺激（side effect），我们随即对外界产生反馈（side effect）。** 上下文、时间无关的函数是易于沉淀、组合的。面向对象是多变而敏感的。 **

在开发App中，我们定义可能的"side effect"作响应源、以函数（数据处理函数、渲染函数，等等）维度来抽象，然后组合，随着某个"side effect"的启动，App的一切就将永远的运转下去。

这里借用cycle的图：

